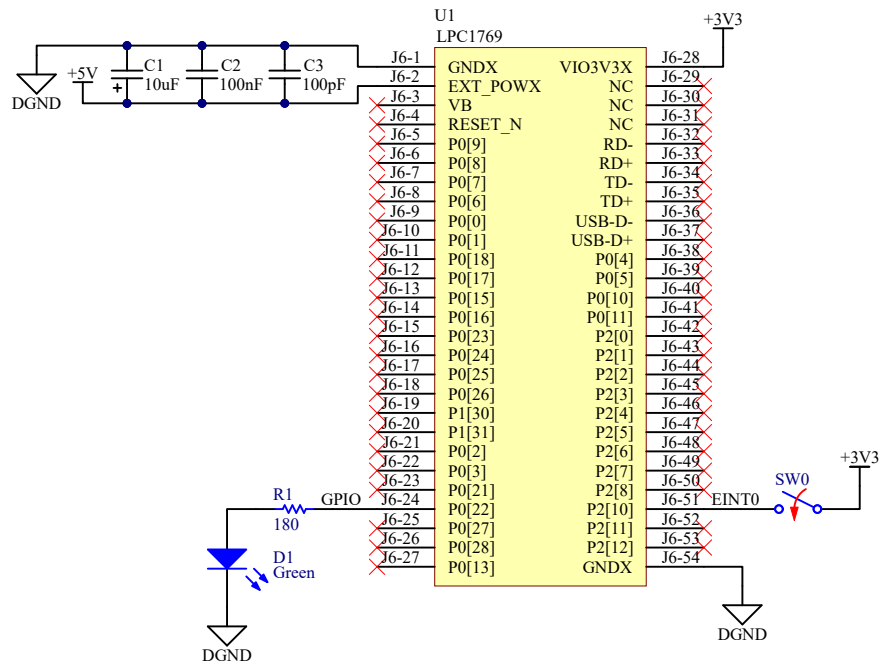


```

1  /*=====
2  * @file    Ejercicio_4_2.c
3  *
4  * @brief    Genera una señal PWM por software mediante SysTick para controlar el
5  *           brillo de un LED en P0.22, modificando el duty cycle vía EINT0.
6  *
7  * @details  - Periodo de la señal PWM: 10 ms (base de tiempo SysTick de 1 ms).
8  *           - Duty cycle inicial: 10% (1 ms alto, 9 ms bajo).
9  *           - EINT0 (P2.10): incrementa el duty cycle en pasos de 10% (1 ms)
10 *             hasta el 100%, reiniciando al 10%.
11 *
12 * @author   Ing. E. Migliore
13 *
14 * @version  1.0
15  */=====

```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES Y MACROS
8  *=====*/
9  #define NVIC_PRIO_BITS      5U          /* Bits físicos de prioridad (0 a 31) */
10
11  /* Identificadores de Prioridad de IRQ */
12  #define SYSTICK_PRIORITY    0U
13  #define EINT0_PRIORITY      1U
14
15  /* Identificadores de IRQ */
16  #define SYSTICK_IRQ         -1
17  #define EINT0_IRQ           18U
18
19  /* Valores de recarga para SysTick */
20  #define N_TICKS_1MS        //COMPLETAR
21
22  #define SYSTICK_CTRL_ON     0x07U
23  #define SYSTICK_CTRL_OFF    0x00U
24
25  #define PWM_PERIOD_STEPS    10U          // 10 pasos de 1 ms = Periodo de 10 ms
26
27  /*=====
28  * DEFINICIÓN DE VARIABLES GLOBALES
29  *=====*/
30  volatile uint8_t dutyCycles = 1;
31
32  /*=====
33  * PROTOTIPOS DE FUNCIONES
34  *=====*/
35  static inline void cfgNVIC(int32_t, uint32_t);
36
37  void cfgGPIO(void);
38  void cfgIntExt(void);
39  void cfgSysTick(uint32_t, uint32_t);
40
41  /*=====
42  * FUNCIÓN PRINCIPAL
43  *=====*/
44  int main(void)
45  {
46      /*-----
47      * INICIALIZACIÓN DEL MCU
48      *-----*/
49      cfgGPIO();
50      cfgIntExt();
51      cfgSysTick(N_TICKS_1MS, SYSTICK_CTRL_ON);
52
53      /*-----
54      * INICIO PROGRAMA PRINCIPAL
55      *-----*/
56      while(1){}
57
58      return 0;
59  }
60
61  /*=====
62  * DEFINICIONES DE FUNCIONES
63  *=====*/
64  /*****
65  * @brief   Configura el NVIC y SCB para la habilitación de interrupciones.
66  *
67  * @param   <irq_number>      Número de IRQ en la tabla del vector.
68  * @param   <priority_number> Nivel de prioridad del vector.
69  *

```

```

70  * @return None
71  *****/
72  static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
73  {
74      //Asigna Prioridad de Interrupción
75      if(irq_number < 0) {
76          //Ejecuta interrupciones del sistema.
77          //SHP = System Handler Priority Registers
78          SCB->SHP[((uint32_t)(irq_number) & 0xF) - 4] = ((priority_number << (8 -
79              NVIC_PRIO_BITS)) & 0xFF);
80      } else {
81          //Ejecuta interrupciones de periféricos.
82          //IP = Interrupt Priority Registers
83          NVIC->IP[(uint32_t)(irq_number)] = ((priority_number << (8 - NVIC_PRIO_BITS)) &
84              0xFF);
85      }
86      //Habilitación de Interrupción
87      NVIC->ISER[((uint32_t)irq_number >> 5)] = (1 << ((uint32_t)irq_number & 0x1F));
88  }
89  /*****
90  * @brief   Configura las direcciones GPIO, modos de función y resistencias.
91  *
92  * @details - P0.22: Salida digital.
93  *           - P2.10: Función EINT0 con Pull-Down interno.
94  *
95  * @param   None
96  *
97  * @return  None
98  *****/
99  void cfgGPIO(void)
100  {
101      LPC_PINCON->PINSEL1  &= ~(0b11 << 12); //P0.22 con Funcionalidad GPIO
102      LPC_PINCON->PINSEL4  |= (0x01 << 20);   //P2.10 con Funcionalidad EINT0
103
104      LPC_GPIO0->FIODIR    |= (1 << 22);      //P0.22 como Salida Digital
105
106      LPC_PINCON->PINMODE4 |= (0b11 << 20);   //P2.10 con Resistencia Pull-Down
107      Habilitada
108
109      return;
110  }
111  /*****
112  * @brief   Configura las interrupciones externas EINT0.
113  *
114  * @details - EINT0 (P2.10): Detección por Flanco Ascendente.
115  *           - Limpia banderas pendientes y asigna prioridades en el NVIC.
116  *
117  * @param   None
118  *
119  * @return  None
120  *****/
121  void cfgIntExt(void)
122  {
123      LPC_SC->EXTMODE  |= (1 << 0);           //EINT0 por Flanco
124      LPC_SC->EXTPOLAR |= (1 << 0);           //EINT0 por Flanco Ascendente
125
126      LPC_SC->EXTINT    |= (0b01 << 0);       //Limpia Flag EINT0
127
128      cfgNVIC(EINT0_IRQ, EINT0_PRIORITY);
129
130      return;
131  }
132
133  /*****
134  * @brief   Configura el módulo SysTick Timer
135  *

```

```

136 * @param <nTicks> Valor de recarga del registro LOAD.
137 * @param <cTicks> Registro de control CTRL (Habilitación/Reloj/IRQ).
138 *
139 * @return None
140 *****/
141 void cfgSysTick(uint32_t nTicks, uint32_t cTicks)
142 {
143     SysTick->LOAD = nTicks; //Carga cuentas en STRELOAD
144     SysTick->VAL = 0; //Carga cuentas a cero STCURR
145     SysTick->CTRL = cTicks; /* Habilita Contador (ENABLE) (1<<0)
146                             * Habilita Interrupcion (TICKINT) (1<<1)
147                             * Selecciona fuente de reloj (CLKSOURCE) (1<<2)
148                             */
149
150     cfgNVIC(SysTick_IRQn, SYSTICK_PRIORITY);
151
152     return;
153 }
154
155 /*****
156 * @brief Rutina de Servicio de Interrupción (ISR) para el vector EINT0.
157 *
158 * @details Incrementa el ancho de pulso (duty cycle) en pasos de 10% (1 ms).
159 * Si alcanza el 100% (10 ms), reinicia el ciclo al 10% (1 ms).
160 *
161 * @param None
162 *
163 * @return None
164 *****/
165 void EINT0_IRQHandler(void)
166 {
167     dutyCycles++;
168     if (dutyCycles > PWM_PERIOD_STEPS) {
169         dutyCycles = 1; //Vuelve al 10% de duty cycle
170     }
171
172     LPC_SC->EXTINT |= (1 << 0); //Limpia Flag EINT0
173
174     return;
175 }
176
177 /*****
178 * @brief Rutina de Servicio de Interrupción (ISR) para el vector SysTick.
179 *
180 * @details Modula el pin P0.22 comparando un contador discreto (0 a 9) con el
181 * valor actual de 'dutyCycles' asignado.
182 *
183 * @param None
184 *
185 * @return None
186 *****/
187 void SysTick_Handler(void)
188 {
189     static uint8_t pwmCounter = 0;
190
191     // Generación de la señal PWM
192     if (pwmCounter < dutyCycles) {
193         LPC_GPIO0->FIOSET = (1 << 22); // Led ON
194     } else {
195         LPC_GPIO0->FIOCLR = (1 << 22); // Led OFF
196     }
197
198     // Incrementa la base de tiempo del PWM y reinicia cada 10 ms
199     pwmCounter++;
200     if (pwmCounter >= PWM_PERIOD_STEPS) {
201         pwmCounter = 0;
202     }
203
204     return;

```

205 }
206
207